Project Planning & Management

E-Commerce Data Engine & AI Assistant with Integrated Data Quality Monitoring

Mohamed Ahmed Abdelghany · Adham Younes Abdelghany · Mahmoud Alaa Elbanna · Moaaz Mohamed Moustafa

Revised draft — 12 October 2026

Document control

Project: E-Commerce Data Engine & AI Assistant with Integrated Data Quality Monitoring

Track: DEPI — Data Engineering

Team: Mohamed Ahmed Abdelghany · Adham Younes Abdelghany · Mahmoud Alaa Elbanna · Moaaz Mohamed Moustafa

Version: Revised draft — 12 October 2026

Status note: Dates and tools below reflect the current project plan and proposed decisions. Confirm programme deadlines and Azure resource availability with the DEPI coordinator before submission.

1. Project Overview

This project builds an end-to-end batch data platform for e-commerce analytics and AI-assisted product discovery. It combines source ingestion, transformation, a dimensional warehouse, business intelligence, review sentiment analysis, a retrieval-augmented generation (RAG) assistant, and an independent Data Quality (DQ) and Trust layer.

The four logical business domains are Orders, Inventory, Shipping, and Payments/Reviews. The public Olist Brazilian E-Commerce dataset supplies order, item, payment, product, seller, customer, review, and delivery-related records. Inventory will be synthetic because Olist does not provide historical stock-on-hand data. Shipping attributes will be derived from available order timestamps, shipping-limit dates, and freight values; derived values must be labelled as derived, not source facts.

Processing is scheduled batch processing, not true streaming. The first implementation should run locally and reproducibly; Azure deployment follows once access and service choices are confirmed.

2. Objectives and measurable success criteria

- O1 — Automated pipelines: Airflow runs the agreed DAG end-to-end without manual intervention; task outcomes, row counts, timestamps, and errors are logged.
- O2 — Analytical warehouse: a documented star schema is loaded successfully; key relationships and required columns pass automated tests.
- O3 — BI dashboard: users can filter by date and category and view top products by units sold and revenue, average item-price trends, and review sentiment.
- O4 — RAG assistant: for a test set of at least 20 representative questions, answers cite or identify retrieved product/review evidence, do not invent missing attributes, and explicitly abstain when the data cannot answer.
- O5 — DQ engine: all documented injected test faults are detected; findings include rule ID, severity, affected table/field, observed value, expected condition, and timestamp.
- O6 — Trust dashboard: displays Healthy, Warning, or Critical status for the data used by BI and the assistant, with recent findings and history.

- O7 — Reproducibility: a new developer can start the local stack from documented setup instructions and environment variables without committed secrets.

3. Scope

In scope: Batch ingestion and transformation; PostgreSQL structured staging; MongoDB review-document storage if retained in the final implementation; star-schema warehouse on the selected Azure service; Airflow orchestration; Docker; Streamlit dashboard (proposed default); Portuguese-capable sentiment analysis; RAG assistant; DQ engine; Trust/Health dashboard; tests and documentation.

Out of scope: True event streaming; customer-facing checkout/storefront; processing real payment transactions; production-grade enterprise security; custom LLM pretraining; automatic silent correction of data-quality failures.

4. Architecture and technology decisions

Processing: Use Pandas as the default transformation engine for the Olist project scale. Spark is optional and should only be added if a measured performance or scalability need justifies its setup. Do not implement the same transformation twice merely to list both tools.

Warehouse/cloud: Target Azure, but select the exact service after checking DEPI access, credits, and team familiarity. Candidate: Azure Database for PostgreSQL for a straightforward relational warehouse; Azure Synapse/Fabric only if available and required. Record the final choice and rationale in the System Analysis & Design document. Keep PostgreSQL local as a development fallback.

Orchestration and packaging: Apache Airflow schedules and retries tasks; Docker Compose should define the local services and their configuration.

BI: Streamlit is the proposed dashboard framework. Confirm the choice with the team and use it consistently in the design and implementation.

RAG: Use structured SQL queries for exact filters and aggregations (budget, category, sales, stock); use semantic retrieval only for text such as review comments or documentation. Select the embedding model, LLM, and vector store after a small evaluation of language support, cost, latency, local/cloud deployment, and reproducibility. Do not commit API keys.

Sentiment: Evaluate a Portuguese-capable model, such as BERTimbau, against a small manually reviewed sample. For reviews with no comment text, use review_score as a clearly labelled proxy (4–5 positive, 3 neutral, 1–2 negative), not as model-predicted text sentiment.

5. Team responsibilities

Mohamed Ahmed Abdelghany: Primary: ETL pipelines and Airflow. Secondary: warehouse loading.

Adham Younes Abdelghany: Primary: warehouse design, Azure, Docker/DevOps. Secondary: ETL.

Mahmoud Alaa Elbanna: Primary: BI dashboard, sentiment analysis, RAG assistant. Secondary: DQ dashboard integration.

Moaaz Mohamed Moustafa: Primary: DQ engine and Trust dashboard. Secondary: testing and QA.

Responsibilities are a proposed allocation; the team should confirm ownership and backup owners in the task board. Each task must have one accountable owner and a due date.

6. Methodology and collaboration

Use one-week Agile-style sprints: planning at the start, a short mid-week unblock meeting, and a review/demo at the end. Track tasks in one shared Trello or Notion board. Use Git feature branches and pull requests reviewed by at least one teammate. Integrate incrementally from the first ETL milestone rather than waiting until the end.

7. Schedule and milestones

Week 1 (3–9 Oct 2026): planning, literature review, requirements, repository and dataset setup.

Week 2 (10–16 Oct): final review and submission of Planning, Literature Review, and Requirements Gathering.

Week 3 (17–23 Oct): architecture, data flow, ERD, and warehouse schema approved.

Week 4 (24–30 Oct): detailed pipeline, RAG, DQ rules, and dashboard wireframes.

Week 5 (31 Oct–6 Nov): finalise System Analysis & Design; start ETL.

Week 6 (7–13 Nov): ETL for domains, Airflow, warehouse load, integration tests.

Week 7 (14–20 Nov): BI dashboard, sentiment, and RAG assistant.

Week 8 (21–27 Nov): DQ engine, Trust dashboard, integration, code freeze target 27 Nov.

Week 9 (28 Nov–4 Dec): fixes, final testing, code submission target 30 Nov, report and presentation target 4 Dec.

These are planning dates from the team draft, not independently verified official DEPI deadlines. Confirm them before submission.

8. Risks and mitigations

R1 — Azure access/credits delayed (Medium likelihood / High impact): Develop and test locally first; keep deployment configuration separate and migrate when access is confirmed.

R2 — Dataset limitations (Medium / Medium): Generate synthetic inventory with a fixed seed; derive shipping metrics from source timestamps; use product IDs/categories rather than invented product names; use a Portuguese-capable sentiment method.

R3 — Spark overhead (Medium / Medium): Use Pandas by default; introduce Spark only after measuring a real need.

R4 — LLM cost/rate limits (Medium / Medium): Set request limits, cache embeddings/results where appropriate, and document a local/open-source fallback.

R5 — Late integration (High / High): Agree schemas and interfaces early; integrate a thin end-to-end path by Week 6.

R6 — Team member unavailable (Low / High): Assign secondary owners and keep runbooks and decisions in the repository.

R7 — DQ false alarms (Medium / Medium): Establish a clean-data baseline, distinguish known baseline issues from new anomalies, and tune warning/critical rules using tests.

R8 — Deadline slip (Medium / High): Target code freeze 27 Nov and reserve the final week for testing, fixes, documentation, and presentation.

9. Data Quality policy and failure behavior

Every check must have a stable rule ID, description, owner, severity, threshold/rationale, and test case. Baseline anomalies should be reported separately from newly introduced anomalies. A Critical schema or reconciliation failure should mark the batch as blocked or quarantined before it reaches trusted downstream consumers; a Warning may allow the batch to continue but must mark affected outputs as degraded. The exact policy must be configurable and documented.

The 1.00 BRL reconciliation tolerance is a proposed project rule based on the draft's stated clean-data baseline (249 of 98,665 orders, about 0.25%, exceeded that tolerance). Recompute this baseline from the actual downloaded files and document exclusions, rounding, cancelled/unavailable orders, missing order items, and the exact formula before treating the threshold as final.

10. Communication, tools, and quality assurance

Communication: WhatsApp; meetings: Google Meet/Zoom; source control: GitHub; task tracking: Trello/Notion; documentation: shared drive and repository.

Planned stack: Python, Pandas, NumPy, scikit-learn/Transformers as needed, PostgreSQL, MongoDB if retained, Airflow, Docker, selected Azure service, Streamlit, and a selected RAG framework/vector store. Mark tools as Planned until actually configured.

QA: peer review each document; unit tests for transformations; integration tests for row counts, keys, joins, and reconciliation; end-to-end DAG test; deliberate fault-injection tests for schema change, currency/price outlier, and inventory mismatch; no secrets in Git.

11. Assumptions and constraints

Olist amounts are in Brazilian reais (BRL); no currency conversion is planned. The source data is historical (2016–2018), so inventory and stock recommendations are a demonstration, not a claim about current real-world availability. Synthetic inventory generation must use a fixed random seed and a documented rule for relating stock to units sold. Team capacity is estimated at 8–10 hours per person per week, subject to confirmation.

12. Submission readiness checklist

Confirm official deadlines and Azure service/access.

Approve Pandas-first processing and confirm Streamlit.

Document the selected LLM, embedding model, vector store, and Portuguese sentiment approach.

Approve the star schema, ERD, data dictionary, and source-to-target mappings.

Set measurable acceptance tests and severity/stop-or-continue behavior.

Add complete references to Literature Review and link requirements to test cases.